

Optimización de Modelos de Aprendizaje Automático mediante Algoritmos Metaheurísticos Bioinspirados: XGBoost, Hibridación GWO–MFO y Redes Neuronales MFO

López Cabrera Lenin Francisco, Jose Mario Camacho Monar, Brayan Gualberto Cárdenas Medina, Dennys Alexander Becerra Gavidia y Dennis Iván Sánchez Palomo

Resumen—La predicción del incumplimiento crediticio exige modelos capaces de aprender a partir de datos tabulares desbalanceados, donde la clase minoritaria concentra el mayor riesgo económico. Este trabajo estudia tres etapas de optimización bioinspirada: primero, un modelo XGBoost optimizado mediante *Grey Wolf Optimizer* (GWO); segundo, una hibridación secuencial GWO–*Moth-Flame Optimization* (MFO); y tercero, una red neuronal multicapa cuyos hiperparámetros son ajustados con MFO. El conjunto de datos contiene 30,000 registros y 23 variables predictoras, divididos de forma estratificada en 24,000 observaciones de entrenamiento y 6,000 de prueba. En XGBoost, GWO con 20 iteraciones elevó el PR-AUC de 0.5579 a 0.5646, mientras que 50 iteraciones alcanzaron el mayor *recall*, 0.6473. El híbrido de 50 iteraciones obtuvo PR-AUC de 0.5640, F1 de 0.5512 y exactitud de 0.7815. En la red neuronal, MFO seleccionó una tasa de aprendizaje de 0.00937, *dropout* de 0.32 y capas ocultas de 75 y 61 neuronas; el modelo final alcanzó ROC-AUC de 0.7673 y exactitud de 0.81, aunque su *recall* para incumplidores fue 0.40. Los resultados muestran que las metaheurísticas permiten ajustar perfiles predictivos distintos y que la selección de la métrica objetivo resulta decisiva en problemas desbalanceados.

Index Terms—XGBoost, GWO, MFO, red neuronal multicapa, optimización de hiperparámetros, riesgo crediticio, clases desbalanceadas.

I. INTRODUCCIÓN

LA detección temprana del incumplimiento crediticio constituye un problema relevante para las instituciones financieras, pues una clasificación incorrecta de un cliente de alto riesgo puede ocasionar pérdidas económicas. El problema presenta dos dificultades principales: los datos son tabulares y la clase de incumplidores es minoritaria. En este escenario, la exactitud puede ofrecer una visión optimista del desempeño, aun cuando el modelo omita una proporción importante de los casos de interés. Por ello, métricas como el área bajo la curva Precision–Recall (PR-AUC), el puntaje F1 y el *recall* son especialmente relevantes [4].

XGBoost es un método de aprendizaje de conjunto basado en árboles de decisión, no una red neuronal. Su rendimiento depende de hiperparámetros que controlan la complejidad, la tasa de aprendizaje, el muestreo y el tratamiento del desbalance [1]. Por otra parte, las redes neuronales multicapa requieren seleccionar su arquitectura, tasa de aprendizaje y

regularización. Ambos tipos de modelos originan espacios de búsqueda no lineales en los que una configuración manual puede resultar insuficiente.

Este trabajo utiliza algoritmos metaheurísticos bioinspirados. Aunque con frecuencia se agrupan informalmente con los algoritmos evolutivos, GWO y MFO no son algoritmos genéticos: no emplean operadores de selección, cruce y mutación. GWO representa el liderazgo y la caza cooperativa de una manada de lobos [2], mientras que MFO modela vuelos en espiral de polillas alrededor de llamas [3]. La investigación se organiza, según la secuencia experimental desarrollada, en: 1) XGBoost optimizado con GWO; 2) hibridación secuencial GWO–MFO; y 3) optimización MFO de una red neuronal multicapa.

Las contribuciones del estudio son las siguientes:

- se evalúa GWO bajo presupuestos de 20 y 50 iteraciones para ajustar XGBoost en un problema crediticio desbalanceado;
- se implementa una estrategia de relevo que combina exploración global con GWO y explotación local con MFO;
- se aplica MFO a la selección de cuatro hiperparámetros de una red neuronal multicapa;
- se analiza el compromiso entre PR-AUC, F1, *recall* y exactitud, evitando equiparar métricas de naturaleza distinta.

II. DATOS Y DISEÑO EXPERIMENTAL

II-A. Conjunto de datos

Se empleó el conjunto *Default of Credit Card Clients*, compuesto por 30,000 observaciones de clientes y una variable objetivo binaria: 0 para clientes que cumplen sus pagos y 1 para clientes que incumplen [5]. Después de eliminar el identificador, se utilizaron 23 variables predictoras. La partición fue estratificada: 80 % para entrenamiento (24,000 registros) y 20 % para prueba (6,000 registros). En el conjunto de prueba se conservaron 4,673 observaciones de la clase mayoritaria y 1,327 de la clase minoritaria.

Para los experimentos con XGBoost se calculó la relación entre ejemplos negativos y positivos,

$$w_+ = \frac{N_0}{N_1} = 3,5206, \quad (1)$$

que sirve como referencia para `scale_pos_weight`. En la red neuronal, las variables se estandarizaron usando únicamente los estadísticos del conjunto de entrenamiento.

II-B. Métricas

Se utilizaron exactitud, *recall*, F1, ROC-AUC y PR-AUC. El *recall* de la clase positiva se define como

$$\text{Recall} = \frac{TP}{TP + FN}, \quad (2)$$

donde TP y FN representan verdaderos positivos y falsos negativos. El puntaje F1 es la media armónica entre precisión y *recall*. En datos desbalanceados se priorizó PR-AUC para XGBoost, pues resume la relación entre precisión y sensibilidad de la clase minoritaria. ROC-AUC se mantuvo para el experimento neuronal porque fue la métrica registrada por ese notebook. Ambas áreas no deben compararse de manera directa.

III. XGBOOST OPTIMIZADO CON GWO

III-A. Modelo base

XGBoost construye aditivamente árboles que corrigen los errores de los árboles previos. En la iteración t , la predicción se expresa como

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i), \quad (3)$$

y el entrenamiento minimiza una función formada por la pérdida predictiva y un término de regularización de la complejidad de los árboles [1].

El modelo base utilizó 500 estimadores, profundidad máxima 6, tasa de aprendizaje 0.05, `subsample=0.8`, `colsample_bytree=0.8` y métrica de evaluación PR-AUC. Este punto de referencia obtuvo PR-AUC de 0.5579, F1 de 0.5392, *recall* de 0.6089 y exactitud de 0.7698.

III-B. Optimización mediante GWO

Cada lobo representa un vector de seis hiperparámetros. Los tres mejores candidatos, denominados α , β y δ , guían al resto de la población. En forma simplificada, la distancia y actualización respecto de un líder se calculan mediante

$$\mathbf{D} = |\mathbf{C}\mathbf{X}_p(t) - \mathbf{X}(t)|, \quad (4)$$

$$\mathbf{X}(t+1) = \mathbf{X}_p(t) - \mathbf{A}\mathbf{D}, \quad (5)$$

donde $\mathbf{A}$ disminuye durante la ejecución para desplazar el comportamiento desde la exploración hacia la explotación [2].

La función objetivo minimizó $1 - \text{PR-AUC}$. La Tabla I presenta el espacio de búsqueda.

Se probaron 20 y 50 iteraciones con una población de 20 lobos. Además, se realizaron 15 ejecuciones independientes y se aplicó la prueba pareada de Wilcoxon sobre PR-AUC. El análisis reportó $p = 0,000031$ para cada configuración GWO frente al modelo base y $p = 0,015076$ entre GWO de 20 y 50 iteraciones.

Tabla I
ESPACIO DE BÚSQUEDA PARA XGBOOST

Hiperparámetro	Inferior	Superior
<code>n_estimators</code>	100	600
<code>max_depth</code>	3	10
<code>learning_rate</code>	0.01	0.15
<code>subsample</code>	0.60	1.00
<code>colsample_bytree</code>	0.60	1.00
<code>scale_pos_weight</code>	2.00	5.00

IV. HIBRIDACIÓN SECUENCIAL GWO-MFO

La hibridación propuesta distribuye el presupuesto computacional en dos fases. Durante la primera mitad, GWO explora todo el dominio de la Tabla I. Después se construye una región local alrededor de la mejor posición encontrada. Para cada dimensión j , el radio de ajuste es

$$r_j = 0,2 (U_j - L_j), \quad (6)$$

y los nuevos límites se restringen al intervalo global. Durante la segunda mitad, MFO explota esta región reducida.

MFO conserva las mejores soluciones como llamas y actualiza cada polilla mediante una espiral logarítmica:

$$\mathbf{X}_i^{t+1} = \mathbf{D}_i e^{b\tau} \cos(2\pi\tau) + \mathbf{F}_j, \quad (7)$$

donde $\mathbf{D}_i$ es la distancia entre la polilla y la llama, b controla la forma de la espiral y τ es un valor aleatorio cuyo intervalo cambia con las iteraciones [3].

La aptitud híbrida integra PR-AUC y F1 mediante una media geométrica y penaliza sensibilidades inferiores a 0.55:

$$S = \sqrt{\text{PR-AUC} \times F1}, \quad (8)$$

$$P = \begin{cases} 0,8(0,55 - R), & R < 0,55, \\ 0, & R \geq 0,55, \end{cases} \quad (9)$$

$$\text{fitness} = (1 - S) + P, \quad (10)$$

donde R es el *recall*. Se evaluaron 20 iteraciones (10 GWO y 10 MFO) y 50 iteraciones (25 GWO y 25 MFO), con 15 agentes.

V. OPTIMIZACIÓN MFO DE UNA RED NEURONAL

El tercer experimento aplica MFO a una red neuronal multicapa (MLP). Esta etapa sí corresponde específicamente a la optimización de redes neuronales. La arquitectura contiene una entrada de 23 variables, dos capas densas ocultas con activación ReLU, normalización por lotes después de la primera capa, regularización *dropout* y una salida sigmoidal para clasificación binaria.

Cada polilla codifica cuatro hiperparámetros: tasa de aprendizaje $\eta \in [10^{-4}, 10^{-2}]$, *dropout* $d \in [0,1,0,5]$, neuronas de la primera capa $n_1 \in [16, 128]$ y neuronas de la segunda capa $n_2 \in [8, 64]$. La aptitud fue $1 - \text{ROC-AUC}_{val}$. Se emplearon 10 polillas y 10 iteraciones; cada candidato se entrenó hasta 15 épocas, con lotes de 256 y parada temprana de paciencia 3.

La mejor configuración registrada fue $\eta = 0,00937$, $d = 0,32$, $n_1 = 75$ y $n_2 = 61$, con ROC-AUC de validación de

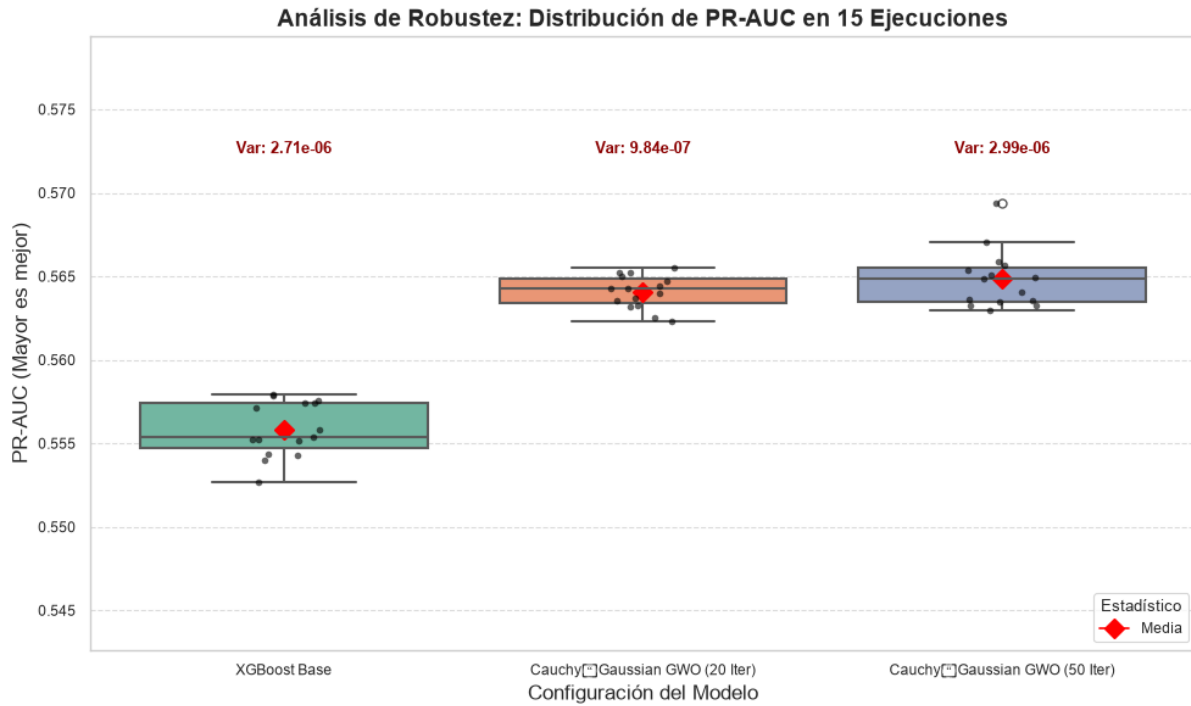


Figura 1. Distribución del PR-AUC en 15 ejecuciones del modelo base y las configuraciones GWO. Los resultados suministrados reportan varianzas del orden de 10^{-6} .

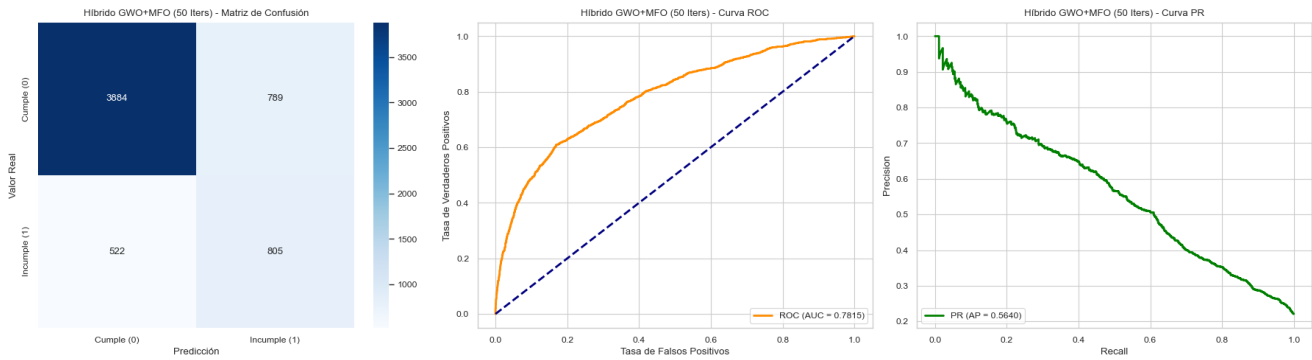


Figura 2. Evaluación del híbrido GWO-MFO con 50 iteraciones: evolución de la aptitud, matriz de confusión, curva ROC y curva Precision-Recall.

0.7814. El modelo definitivo se entrenó hasta 30 épocas con paciencia 5 y restauración de los mejores pesos. En prueba obtuvo ROC-AUC de 0.7673 y exactitud de 0.81. Para la clase incumplidora alcanzó precisión de 0.63, *recall* de 0.40 y F1 de 0.49.

VI. RESULTADOS Y DISCUSIÓN

VI-A. Comparación de XGBoost y la hibridación

La Tabla II resume los experimentos evaluados con PR-AUC. GWO con 20 iteraciones obtuvo el mayor PR-AUC, 0.5646, equivalente a una mejora relativa de 1.20 % frente al modelo base. GWO con 50 iteraciones desplazó el modelo hacia una mayor sensibilidad: detectó 859 incumplidores, frente a 808 del modelo base, y redujo los falsos negativos de 519 a 468. Esta ganancia elevó el *recall* a 0.6473, aunque redujo la exactitud a 0.7453.

En la hibridación, el aumento de 20 a 50 iteraciones redujo la aptitud final de 0.4451 a 0.4425. El PR-AUC aumentó de 0.5606 a 0.5640, F1 de 0.5492 a 0.5512 y la exactitud de 0.7765 a 0.7815. Sin embargo, el *recall* descendió de 0.6157 a 0.6066. Por tanto, el mayor presupuesto produjo un modelo globalmente más equilibrado, pero no el más sensible. La configuración adecuada depende del costo relativo de falsos positivos y falsos negativos.

VI-B. Interpretación del experimento MFO-MLP

La red MFO-MLP alcanzó una exactitud mayor que las configuraciones XGBoost mostradas. No obstante, su *recall* de 0.40 indica que omitió aproximadamente 60 % de los incumplidores. La diferencia se explica, en parte, por la función objetivo: la red maximizó ROC-AUC sin una penalización explícita por baja sensibilidad, mientras que XGBoost incorporó ponderación de clase y, en el híbrido, una penalización de

Tabla II
RESULTADOS DE XGBOOST Y LA HIBRIDACIÓN GWO-MFO

Modelo	Iteraciones	PR-AUC	F1	Recall	Exactitud
XGBoost base	–	0.5579	0.5392	0.6089	0.7698
XGBoost-GWO	20	0.5646	0.5361	0.6104	0.7663
XGBoost-GWO	50	0.5630	0.5293	0.6473	0.7453
XGBoost-GWO-MFO	20	0.5606	0.5492	0.6157	0.7765
XGBoost-GWO-MFO	50	0.5640	0.5512	0.6066	0.7815

recall. Además, no se dispone del PR-AUC de la MLP, por lo que no es válido afirmar que supera a XGBoost en detección de la clase minoritaria.

Estos resultados confirman que el optimizador no puede separarse de la métrica elegida. MFO encontró una arquitectura con buena discriminación global, pero el umbral de 0.5 y la ausencia de una función de costo sensible al desbalance limitaron la detección de incumplidores. Una continuación natural consiste en optimizar PR-AUC o una función multiobjetivo, introducir pesos de clase y ajustar el umbral de decisión.

VII. AMENAZAS A LA VALIDEZ

Los experimentos disponibles presentan limitaciones que deben considerarse antes de una publicación definitiva. Primero, los notebooks de XGBoost emplean el conjunto de prueba como conjunto de evaluación durante la búsqueda y la parada temprana; esto puede introducir optimismo en las métricas finales. Se recomienda reservar un conjunto de validación interno o utilizar validación cruzada anidada y evaluar el conjunto de prueba una sola vez.

Segundo, los resultados GWO del informe Markdown y algunas salidas almacenadas en los notebooks no coinciden exactamente, posiblemente por semillas, variantes del optimizador o ejecuciones distintas. Este manuscrito utiliza como fuente principal el análisis consolidado suministrado para GWO y las salidas registradas para el híbrido y MFO. Para garantizar reproducibilidad deben fijarse semillas, versiones de bibliotecas, presupuesto medido en evaluaciones de la función objetivo y un único protocolo experimental.

Tercero, la significancia estadística reportada se limita a GWO. El híbrido y la MLP requieren múltiples ejecuciones independientes, intervalos de confianza y pruebas pareadas bajo las mismas particiones. Finalmente, comparar presupuestos solo por número de iteraciones no es completamente justo cuando cambian el tamaño de la población y el costo de entrenamiento de cada modelo.

VIII. CONCLUSIONES

Este trabajo presentó tres etapas de optimización bioinspirada aplicadas a la predicción de incumplimiento crediticio. GWO mejoró de forma modesta pero estadísticamente significativa el PR-AUC de XGBoost y permitió ajustar el perfil del clasificador hacia una mayor sensibilidad. La configuración GWO de 50 iteraciones obtuvo el mayor *recall*, mientras que el híbrido GWO-MFO de 50 iteraciones produjo el mejor equilibrio entre PR-AUC, F1 y exactitud dentro de la estrategia secuencial.

MFO también fue capaz de seleccionar automáticamente la tasa de aprendizaje, el *dropout* y el número de neuronas de una MLP. Sin embargo, la red optimizada para ROC-AUC mantuvo un *recall* insuficiente en la clase de incumplidores. En consecuencia, la contribución principal no es únicamente la aplicación de una metaheurística, sino la evidencia de que la función de aptitud determina el comportamiento operativo del modelo.

Como trabajo futuro se propone unificar el protocolo de evaluación, usar validación cruzada anidada, comparar todos los métodos bajo el mismo número de evaluaciones, incorporar Optuna como referencia bayesiana y formular una optimización multiobjetivo que considere simultáneamente PR-AUC, *recall*, costo computacional y estabilidad.

AGRADECIMIENTOS

Los autores expresan su agradecimiento a la Universidad Nacional de Chimborazo (UNACH) por el apoyo brindado durante el desarrollo de esta investigación. De manera especial, agradecen al Ing. Marco Javier Castelo Cabay por su orientación, acompañamiento y valiosos aportes académicos.

REFERENCIAS

- [1] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.
- [2] S. Mirjalili, S. M. Mirjalili, and A. Lewis, "Grey wolf optimizer," *Advances in Engineering Software*, vol. 69, pp. 46–61, 2014, doi: 10.1016/j.advengsoft.2013.12.007.
- [3] S. Mirjalili, "Moth-flame optimization algorithm: A novel nature-inspired heuristic paradigm," *Knowledge-Based Systems*, vol. 89, pp. 228–249, 2015, doi: 10.1016/j.knsys.2015.07.006.
- [4] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, Art. no. e0118432, 2015, doi: 10.1371/journal.pone.0118432.
- [5] I.-C. Yeh and C.-H. Lien, "The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients," *Expert Systems with Applications*, vol. 36, no. 2, pp. 2473–2480, 2009, doi: 10.1016/j.eswa.2007.12.020.
- [6] N. Van Thieu and S. Mirjalili, "MEALPY: An open-source library for latest meta-heuristic algorithms in Python," *Journal of Systems Architecture*, vol. 139, Art. no. 102871, 2023, doi: 10.1016/j.sysarc.2023.102871.